„Other questions"

I. ①. HT, m pos, open addr., n elems. Can $n > m$?

> In open addressing, all elems are stored directly in the HT.
> Each position stores at most 1 elem ⇒ $n \leq m$

[a] Not possible, no matter how the h is defined.

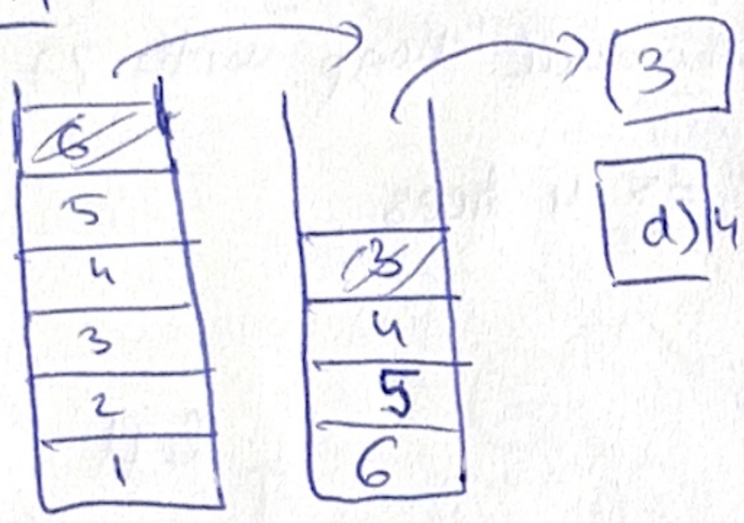
② HT, m pos, open addr, n elems. Search (e); more than n pos checked?

[b] only if ∃(e).

③. BST. instead of x?

[d] 26

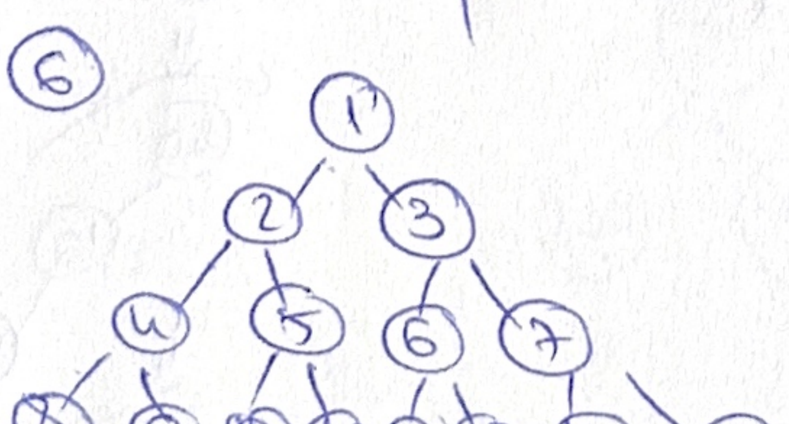
④

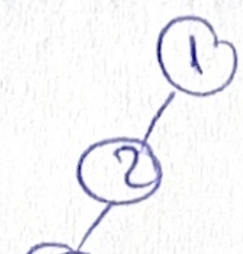


⑤. 4 6 7 + 1 − 2 5 * + +

Token	stack
4	4
6	4 6
7	4 6 7
+	4, 13
1	4, 13, 1
−	4, 12
2	4, 12, 2
5	4, 12, 2, 5
*	4, 12, 10
+	4, 22
+	26

[e] > 15





⇓

$H_{min} = 4$

⇓

$H_{max} = 20 - 1 = 19$

$\Rightarrow H_{max} - H_{min} = 19 - 4 = 15$ [b]

(7). DS for ADTList if we frequently getElement() from pos p? (best)

[a] dyn array

b), c) → SLL, DLL → need traversals $\Theta(n)$

d) → skip list → not designed for positions

e) Bin. heap → not a list

f) HT → no pos.

(8) $12m^3 + n\log_2 m + 52$

a) $O(m^4)$

d) $\Omega(m^3)$

e) $\Omega(1)$

(9) How many binomial trees in a binomial heap with 27 elem?

$27 = 2^4 + 2^3 + 2^1 + 2^0 \Rightarrow 0,1,3,4 \Rightarrow 4$ trees.

(10) preorder: root - left - right

~~DABCE~~ DABCEHFGNI

postorder: left - right - root

ECBAFNIGHD

inorder: left - root - right

BECADFHNGI

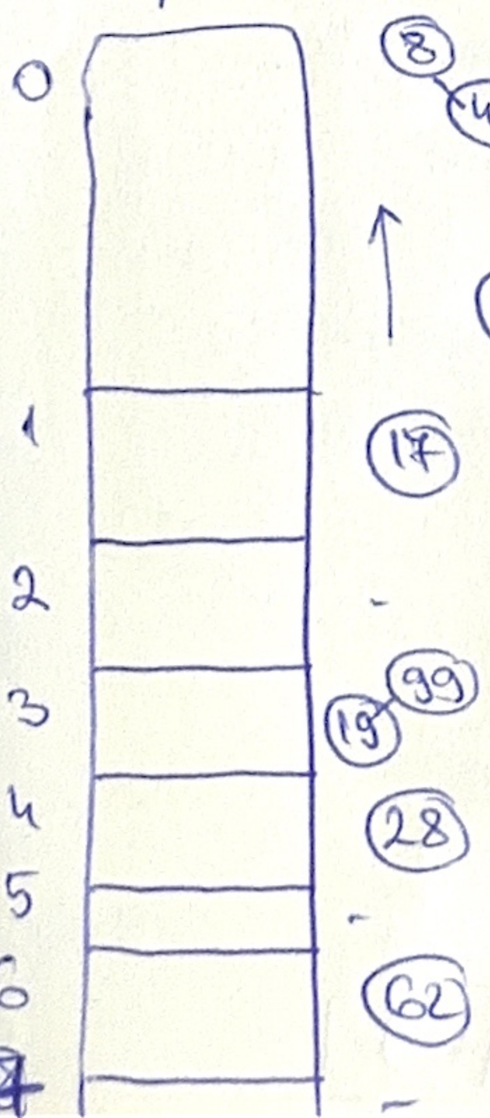
T	left	right
root (pre)	root (IN)	root (post)

(11) HT, $m = 8$, sep ch, div method, root AVL. (instead of a LL).

add 8, 99, 40, ...

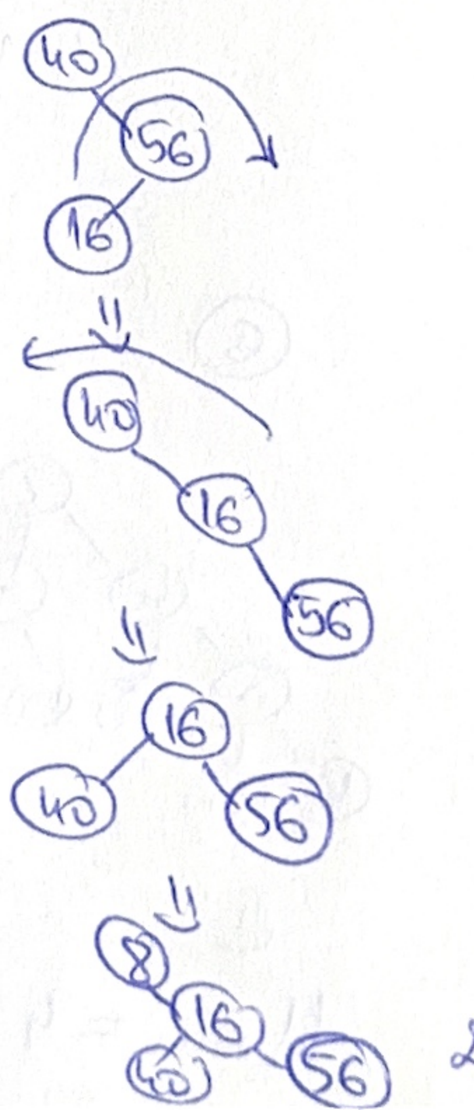
e	8	99	40	19	56	16	17	28	62
h(e)	0	3	0	3	0	0	1	4	6

$\alpha = \frac{n}{m} = \frac{9}{8}$ load factor



unbalanced,
BF = 0 - 3 => heavy-right
(8)

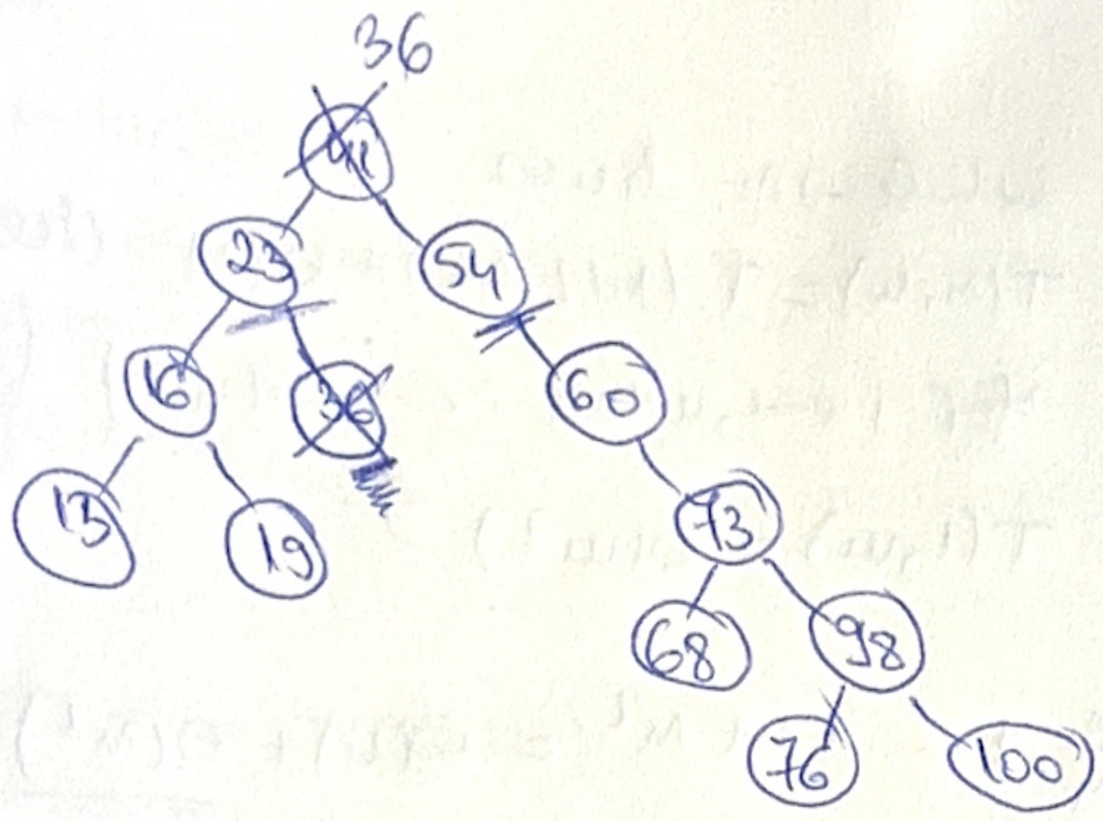
BF(40) = 0 - 2 = -2 => balanced, heavy-right } RL
BF(56) = 1 - 0 = 1 => balanced, heavy-left
BF(16) = 0 - 0 = 0 => balanced.



2

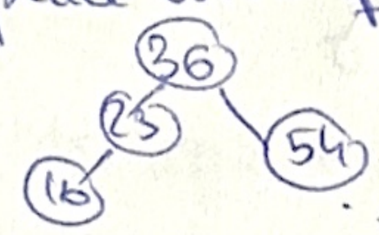
17 June 2024 → 55

7 (10) ≤ BST

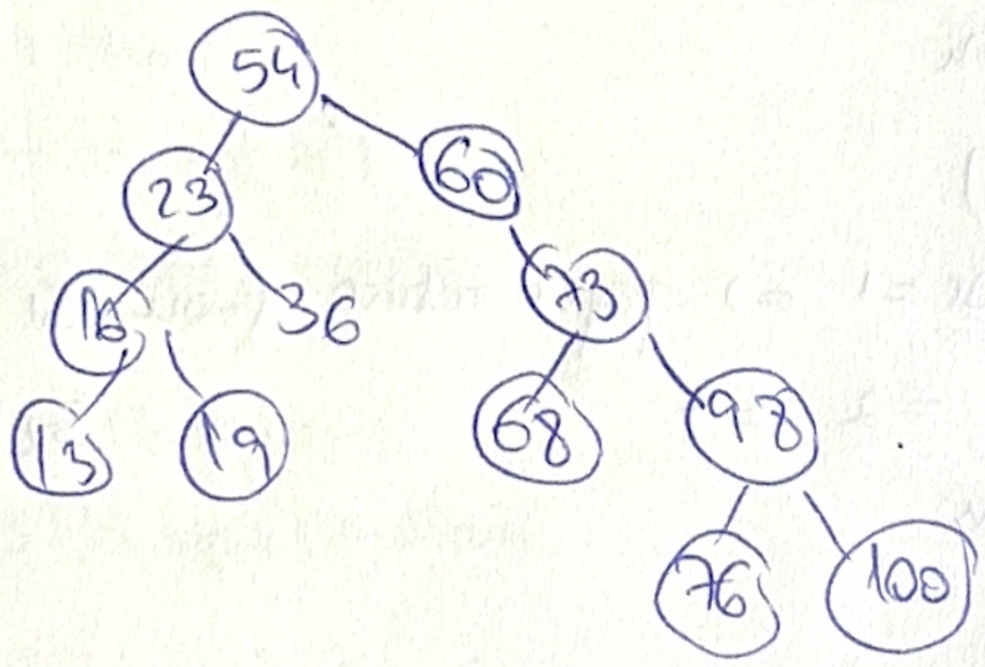


remove 41 => replace with predecessor (max value in its left subtree)

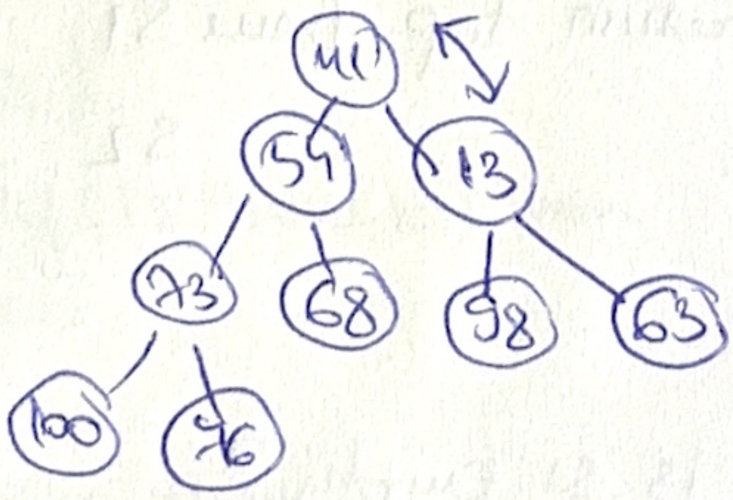
36.



=> replace with successor (min value in its right subtree)

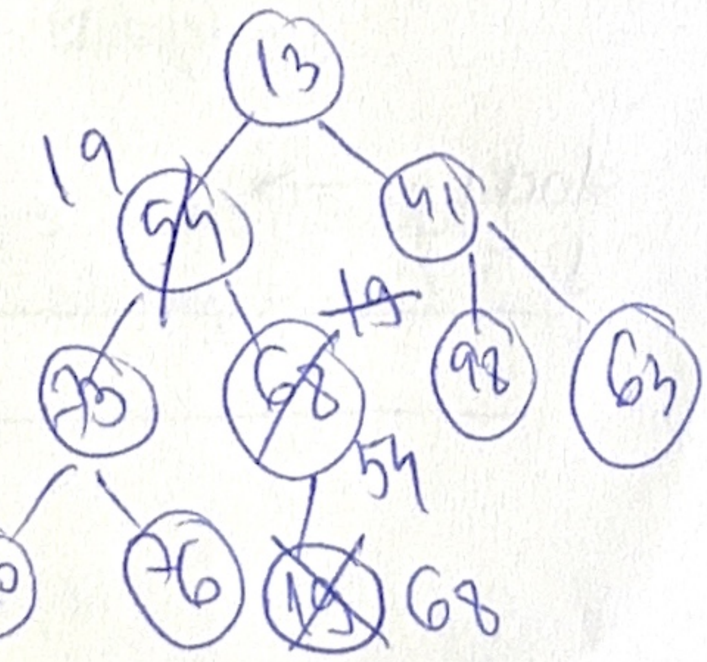


(11) Bin. heap?

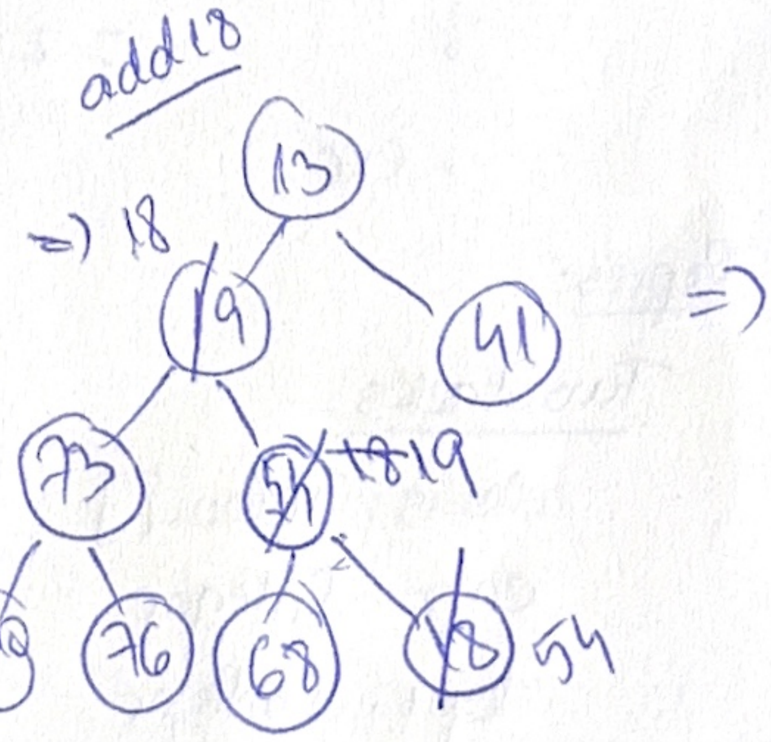


not a ~~bstm~~ heap

add 19

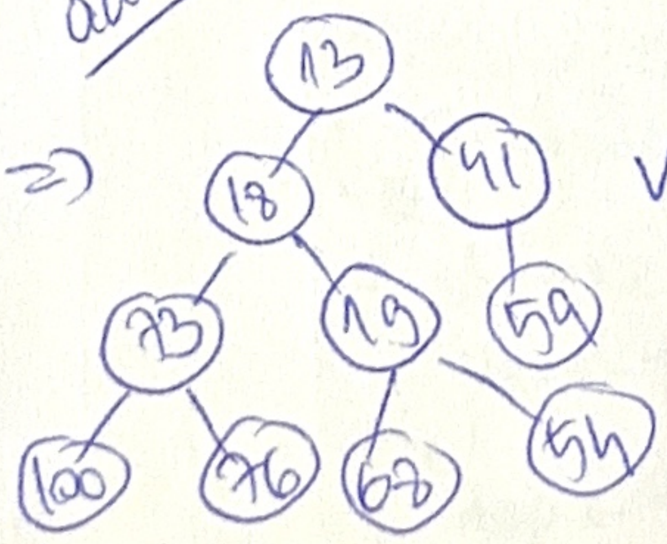


add 18



=>

add 59



✓

(12)

op(n,m)
  k = n/2
  i = 3
    i = 3+n
    n = n-1

while → n times
$T(n,m) = T(n/2,m) + \Theta(n)$ (recursive)
for i ← 1, m*m ← $\Theta(m^2)$ base case
$T(1,m) = \Theta(m^2)$ ↑

$$T(n,m) = n + \frac{n}{2} + \frac{n}{4} + \dots + 1 + m^2 = \Theta(n) + \underline{\Theta(m^2)} = \Theta(m^2)$$

(13) ADT TwoStacks : O(1) complexity

push (ts, elem, stackNr)
- stackNr = 1 ⇒ push to S1
- = 2 ⇒ S2
- exception

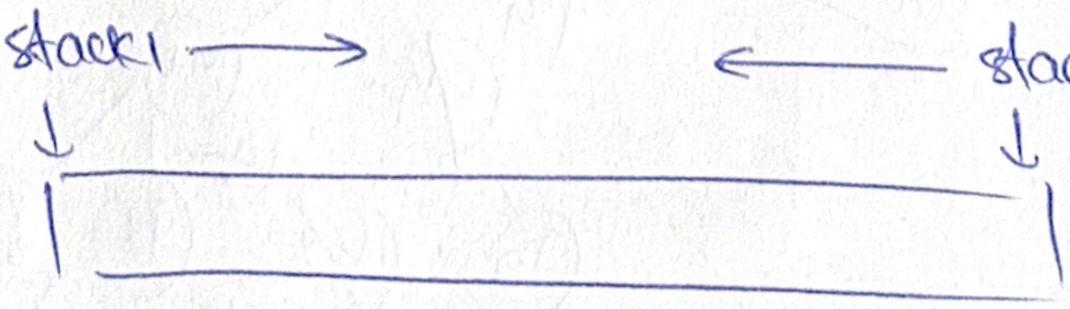
pop (ts, stackNr)
- stackNr = 1 ⇒ pop + return from S1
- = 2 ⇒ S2
- exception

top (ts, stackNr)
- stackNr = 1 ⇒ return top from S1
- = 2 ⇒ S2
- exception

isEmpty (ts, stackNr)
- stackNr = 1 ⇒ is S1 empty?
- = 2 ⇒ S2
- exc.

Repres:

TwoStacks:
  elems: TElem[]
  cap: Integer
  top1: Integer
  top2: Integer



```
init(ts) is
    ts.cap ← initialCap
    ts.elems ← allocate(ts.cap)
    ts.top1 ← -1
    ts.top2 ← ts.cap

push(ts, elem, stackNr)
    if stackNr ≠ 1 and stackNr ≠ 2 then
        @throw...
    if ts.top1 + 1 = ts.top2 then
        resize(ts)
    if stackNr = 1 then
        ts.top1 ← ts.top1 + 1
        ts.elems[ts.top1] ← elem
    else
        ts.top2 ← ts.top2 - 1
        ts.elems[ts.top2] ← elem

pop(ts, stackNr)
    if isEmpty(ts, stackNr) then
        @throw...
    if stackNr = 1 then
        elem ← ts.elems[ts.top1]
        ts.top1 ← ts.top1 - 1
    else
        elem ← ts.elems[ts.top2]
        ts.top2 ← ts.top2 + 1

top(ts, stackNr)
    if isEmpty(ts, stackNr) then
        @throw
    if stackNr = 1 then
        top ← ts.elems[ts.top1]
    else
        top ← ts.elems[ts.top2]
```

```
function isEmpty (ts, stackNr)
    if stackNr = 1 then
        isEmpty ← ts.top1 = -1
    else if stackNr = 2 then
        isEmpty ← ts.top2 = ts.cap
    else
        @ throw
```

$\Theta(1)$ because usually we only change one index and assign one value and reSize is rare.

WC: $\Theta(n)$ (reSize)

amortized: $\Theta(1)$

(14). Find max value in BT of integers

BT:

root: ↑BTNode

Node:

info: Int

left: ↑Node

right: ↑Node

```
function max(bt)
    maxValue ← -1
    traverse(bt.root, maxValue)
    max ← maxValue

subalg traverse(node, maxValue)
    if node ≠ Nil then
        if ↑(node).info > maxValue then
            maxValue ← ↑(node).info
        traverse(↑(node).left, maxValue)
        traverse(↑(node).right, maxValue)
```

$\Theta(n)$ visit all nodes

(15) Quantiler –